

C'est qui ce ZIG ?

Didier Plaindoux



@dplaindoux@functional.cafe



C'est qui ce zigue ?

Systeme Distribué et modèle Acteur (Akawan Kaptngo)

Systeme Expert en Prolog (Crédifix)

Fonctions et Calcul des Ambiants (Ephel)

Libraries OCaml avec des amis (CargoCut)





Langage de programmation système, moderne et robuste
qui propose de la méta-programmation via l'exécution de
code à la compilation !



Langage de programmation système, moderne et robuste
qui propose de la méta-programmation via l'exécution de
code à la compilation !

Langage sans flux de contrôle caché



Langage de programmation système, moderne et robuste
qui propose de la méta-programmation via l'exécution de
code à la compilation !

Langage sans flux de contrôle caché

Langage sans allocations mémoire cachées



Langage de programmation système, moderne et robuste
qui propose de la méta-programmation via l'exécution de
code à la compilation !

Langage sans flux de contrôle caché

Langage sans allocations mémoire cachées

Langage sans préprocesseur, ni macros

Survol du langage ZIG



Types natifs

Types natifs

Nombres (entier et flottant)

i8, u8, i16, u16, i32, u32, i64, u64, isize, usize

f16, f32, f64, f80, f128

Types natifs

Nombres (entier et flottant)

i8, u8, i16, u16, i32, u32, i64, u64, isize, usize

f16, f32, f64, f80, f128

Compatibilité ABI avec C

c_char, c_short, c_ushort, c_int, c_uint, c_long etc.

Types natifs

Nombres (entier et flottant)

i8, u8, i16, u16, i32, u32, i64, u64, isize, usize

f16, f32, f64, f80, f128

Compatibilité ABI avec C

c_char, c_short, c_ushort, c_int, c_uint, c_long etc.

Autres ...

bool, void, anyopaque, noreturn, type, anyerror ...

Optional

?T pour stocker la valeur **null** ou une valeur de type **T**

```
const a : ?u32 = ...;
```

```
const value1 = a or else ...;
```

```
const value2 = a or else unreachable; // panique si null
```

```
const value3 = a?;
```

Optional

?T pour stocker la valeur **null** ou une valeur de type **T**

```
const a : ?u32 = ...;
```

```
if (a != null) {  
    ... // Appel sûr à a.  
} else {  
    ...  
}
```

Optional

?T pour stocker la valeur **null** ou une valeur de type **T**

```
const a : ?u32 = ...;
```

```
if (a) |value| {  
    ...  
} else {  
    ...  
}
```

Optional

?T pour stocker la valeur **null** ou une valeur de type **T**

```
var a : ?u32 = ...;
```

```
if (a) |*value| {  
    ...  
} else {  
    ...  
}
```


Énumération

Erreur

Fonction

```
fn nom(p1: t1, ..., p1: tn) r {  
    ...  
}
```

Les paramètres sont passés par valeur:

Fonction

```
fn nom(p1: t1, ..., p1: tn) r {  
    ...  
}
```

Les paramètres sont passés par valeur:

- les primitives sont copiées et

Fonction

```
fn nom(p1: t1, ..., p1: tn) r {  
    ...  
}
```

Les paramètres sont passés par valeur:

- les primitives sont copiées et
- pour les données complexes cela dépend

Fonction

Possibilité de retourner des erreurs

```
fn nom(p1: t1, ..., p1: tn) !r {  
    ...  
}
```

Fonction

Possibilité de retourner des erreurs connues

```
fn nom(p1: t1, ..., p1: tn) error{}!r {  
    ...  
}
```

Tableaux

```
const hello : [5]u8 = [_]u8{ 'h', 'e', 'l', 'l', 'o' };
```


Contrôle

If conditional selection

```
if (condition) true_branch [else false_branch]
```

Contrôle

For loop

```
const items = [_i32{ 4, 5, 3, 4, 0 }];
```

```
for(items) |item| { ... }
```

```
for(items) |item, index| { ... }
```

```
for(items,items) |item1, item2| { ... }
```

```
for(items) |item| { ... } else { ... }
```

Contrôle

While loop

```
var i : usize = 0;
```

```
while(i < 10) { ... }
```

```
while (i < 10) : (i += 1) {}
```

```
while (eventuallyNullSequence()) |value| { ... }
```

Tout est expression

Structure

Type produit hérité du langage C

Structure

Type produit hérité du langage C

```
const Point = struct {  
    x: i32,  
    y: i32,  
};
```

Structure

Type produit hérité du langage C

```
const Point = struct {  
    x: i32,  
    y: i32,  
};
```

```
fn init(x: i32, y: i32) Point {  
    return Point { .x = x, .y = y };  
}
```

Structure

Type produit hérité du langage C

```
const Point = struct {  
    x: i32,  
    y: i32,  
};
```

```
fn init(x: i32, y: i32) Point {  
    return Point { .x = x, .y = y };  
}
```


Structure

Type produit hérité du langage C

```
const Point = struct {  
    x: i32,  
    y: i32,  
};
```

```
fn init(x: i32, y: i32) Point {  
    return .{ .x = x, .y = y };  
}
```

Structure

Colocaliser représentation et comportements

```
const Point = struct {  
    x: i32,  
    y: i32,  
  
    fn init(x: i32, y: i32) Point {  
        return .{ .x = x, .y = y };  
    }  
};
```

Structure

Typage et référence au type courant

```
const Point = struct {  
    x: i32,  
    y: i32,  
  
    fn init(x: i32, y: i32) @This() {  
        return .{ .x = x, .y = y };  
    }  
};
```

Structure

Structure

Tout fichier source est une Structure

Structure

Tout fichier source est une Structure

x: i32,

Structure

Tout fichier source est une Structure

```
x: i32,
```

```
y: i32,
```

```
fn init(x: i32, y: i32) @This() {
```

Structure

Tout fichier source est une Structure

```
x: i32,  
y: i32,  
  
fn init(x: i32, y: i32) @This() {  
    return .{ .x = x, .y = y };  
}
```


Structure

Tout fichier source est une Structure

x: i32,

y: i32,

```
fn init(x: i32, y: i32) @This() {  
    return .{ .x = x, .y = y };  
}
```

Union

Type somme hérité du langage C

```
const Number = union {  
    integer: i64,  
    float: f64,  
};
```

Soit un integer soit un float

Union

Accès sécurisé à la compilation

```
const Number = union { integer: i64, float: f64, };
```

```
test "Qui ne compile pas :)" {  
    const n = NumberSimple{ .integer = 42 };  
    try std.testing.expectEqual(42, n.float);  
}
```

error: access of union field 'float' while field 'integer' is active

Union

Accès sécurisé à l'exécution

```
const Number = union { integer: i64, float: f64, };
```

```
fn integer(n: Number) i64 {  
    return n.integer;  
}
```

panic: access of union field 'integer' while field 'float' is active

Union Étiquetée

Union Étiquetée

Inspirée des langages fonctionnels

Union Étiquetée

Inspirée des langages fonctionnels

- Etiquetage Implicite

Union Étiquetée

Inspirée des langages fonctionnels

- Etiquetage Implicite**
- Etiquetage Explicite**

Union Étiquetée

Inspirée des langages fonctionnels

- Etiquetage Implicite**
- Etiquetage Explicite**
- Introduction du Filtrage de Motif**

Union

Étiquetage implicite

```
const Number = union(enum) { integer: i64, float: f64, };
```

```
fn zeroInteger() Number {  
    return .{ .integer = 0 };  
}
```

```
fn zeroFloat() Number {  
    return .{ .float = 0 };  
}
```

Union

Étiquetage explicite

```
const Label = enum { entier, flottant };
```

```
const Number = union(Label) { entier: i64, flottant: f64, };
```

```
fn zero(label: Label) Number {  
    return if (label == .entier) .{ .entier = 0 }  
        else .{ .flottant = 0 };  
}
```

Union

Introduction du Filtrage de Motif

```
const Number = union(enum) { entier: i64, flottant: f64, };
```

```
fn increment(number: Number) Number {  
    return switch (number) {  
        .entier => |n| .{ .entier = n + 1 },  
        .flottant => |f| .{ .flottant = f + 1 },  
    };  
}
```

Union

Colocaliser représentation et comportements

```
const Number = union(enum) {  
    entier: i64,  
    flottant: f64,  
  
    fn increment(number: Number) Number {  
        return ...  
    }  
};
```

Union

Typage et référence au type courant

```
const Number = union(enum) {  
    entier: i64,  
    flottant: f64,  
  
    fn increment(self: @This) @This {  
        return ...  
    }  
};
```

Union

Cas des types récurifs

```
const Naturel = union(enum) {  
    Zero,  
    Succ: Naturel,  
};
```

```
error: type 'Naturel' depends on itself for field declared here  
    Succ: Naturel,  
          ^~~~~~
```

Il faut passer par les pointeurs ... comme en Rust !

Pointeur

```
fn zero() u32 {  
    return 0;  
}
```


Pointeur

```
fn zero() u32 {  
    return 0;  
}  
  
export fn main() void {  
    _ = (&zero).*();  
  
}
```

Pointeur et opération implicite

```
fn zero() u32 {  
    return 0;  
}  
  
export fn main() void {  
    _ = (&zero).*();  
    //      ^^ peut être omis (parfois)  
}
```

Pointeur et opération implicite

```
fn zero() u32 {  
    return 0;  
}  
  
export fn main() void {  
    _ = (&zero)();  
  
}
```

Que donne le code assembleur ?

Gestion de la mémoire dans ZIG



Pile ou Tas ?

Allocation sur la Pile

Allocation sur la Pile

```
const Point = struct { x: i32, y: i32 };
```

Allocation sur la Pile

```
const Point = struct { x: i32, y: i32 };
```

```
pub fn main() void {  
    const p1 = Point { .x = 0, .y = 0};  
}
```


Allocation sur la Pile

```
const Point = struct { x: i32, y: i32 };

pub fn main() void {
    const p1 = Point { .x = 0, .y = 0 };
    {
        const p2 = Point { .x = 42, .y = 42 };
        ...
    } // -> p2 fin de portée donc libéré
```

Allocation sur la Pile

```
const Point = struct { x: i32, y: i32 };

pub fn main() void {
    const p1 = Point { .x = 0, .y = 0 };
    {
        const p2 = Point { .x = 42, .y = 42 };
        ...
    } // -> p2 fin de portée donc libéré
    ...
}
```

Allocation dans le Tas

Oui mais lequel ?

Allocation dans le Tas

Oui mais lequel ?

- DebugAllocator

Allocation dans le Tas

Oui mais lequel ?

- **DebugAllocator**
- **SafeAllocator**

Allocation dans le Tas

Oui mais lequel ?

- **DebugAllocator**
- **SafeAllocator**
- **ArenaAllocator**

Allocation dans le Tas

Oui mais lequel ?

- **DebugAllocator**
- **SafeAllocator**
- **ArenaAllocator**
- **PageAllocator**

Allocation dans le Tas

Oui mais lequel ?

- **DebugAllocator**
- **SafeAllocator**
- **ArenaAllocator**
- **PageAllocator**
- ...

Allocation dans DebugAllocator

Allocation dans DebugAllocator

```
const std = @import("std");
```

Allocation dans DebugAllocator

```
const std = @import("std");
```

```
pub fn main() !void {  
    var gpa = std.heap.DebugAllocator(.{}).{};  
    defer _ = gpa.deinit();  
}
```

Allocation dans DebugAllocator

```
const std = @import("std");
```

```
pub fn main() !void {  
    var gpa = std.heap.DebugAllocator(.{}).{};  
    defer _ = gpa.deinit();  
    const allocator = gpa.allocator();
```

Allocation dans DebugAllocator

```
const std = @import("std");
```

```
pub fn main() !void {  
    var gpa = std.heap.DebugAllocator(.{}){};  
    defer _ = gpa.deinit();  
    const allocator = gpa.allocator();  
  
    const array = try allocator.alloc(u8, 10);
```

Allocation dans DebugAllocator

```
const std = @import("std");

pub fn main() !void {
    var gpa = std.heap.DebugAllocator(.{}){};
    defer _ = gpa.deinit();
    const allocator = gpa.allocator();

    const array = try allocator.alloc(u8, 10);
    defer allocator.free(list);
    ...
}
```

“Use-After-Free”

“Use-After-Free”

```
const std = @import("std");
```

```
test "Use after free" {  
    var heap = std.heap.DebugAllocator(.{}).{};  
    defer _ = heap.deinit();
```


“Use-After-Free”

```
const std = @import("std");
```

```
test "Use after free" {  
    var heap = std.heap.DebugAllocator(.{}){};  
    defer _ = heap.deinit();  
    const allocator = heap.allocator();
```

“Use-After-Free”

```
const std = @import("std");
```

```
test "Use after free" {  
    var heap = std.heap.DebugAllocator(.{}){};  
    defer _ = heap.deinit();  
    const allocator = heap.allocator();  
  
    const score_ptr = try allocator.create(u32);
```

“Use-After-Free”

```
const std = @import("std");
```

```
test "Use after free" {  
    var heap = std.heap.DebugAllocator(.{}){};  
    defer _ = heap.deinit();  
    const allocator = heap.allocator();  
  
    const score_ptr = try allocator.create(u32);  
    allocator.destroy(score_ptr);  
}
```

“Use-After-Free”

```
const std = @import("std");

test "Use after free" {
    var heap = std.heap.DebugAllocator(.{}){};
    defer _ = heap.deinit();
    const allocator = heap.allocator();

    const score_ptr = try allocator.create(u32);
    allocator.destroy(score_ptr);

    score_ptr.* = 666; // Use-After-Free here
```

“Use-After-Free”

```
const std = @import("std");

test "Use after free" {
    var heap = std.heap.DebugAllocator(.{}){};
    defer _ = heap.deinit();
    const allocator = heap.allocator();

    const score_ptr = try allocator.create(u32);
    allocator.destroy(score_ptr);

    score_ptr.* = 666; // Use-After-Free here
}
```

“Use-After-Free”

“Use-After-Free”

Execution avec le DebugAllocator :

Segmentation fault at address 0x109a20000

use-afer-free.zig:13:5: 0x1010c10c1 in main (use-afer-free)

```
    score_ptr.* = 666;
```

^

“Use-After-Free”

Execution avec le DebugAllocator :

Segmentation fault at address 0x109a20000

use-afer-free.zig:13:5: 0x1010c10c1 in main (use-afer-free)

```
    score_ptr.* = 666;
```

^

Solutions idiomaticues :

- utilisation du defer ou
- typer avec un optional et mettre le pointeur à null.

“Stack Buffer Overflow”

```
#include <string.h>
```

```
void foo(char* bar) {  
    char c[12];  
    strcpy(c, bar); // pas de vérification  
}
```

**Ecriture à une adresse mémoire de la pile d'appel
située en dehors de la structure de données prévue**

“Stack Buffer Overflow”

```
const std = @import("std");
```

```
fn foo(input: []const u8) void {  
    var buffer: [8]u8 = undefined;  
    @memcpy(buffer[0..], input);  
}
```

```
pub fn main() void {  
    foo("Plus de 8 caractères!"); // 22 caractères  
}
```

“Stack Buffer Overflow”

Execution interrompue :

```
thread 1332888 panic: ... arguments have non-equal lengths
stack-buffer-overflow.zig:5:19: 0x10230955f in foo
@memcpy(buffer[0..], input);
               ^
stack-buffer-overflow.zig:9:8: 0x102309314 in main
foo("Plus de 8 caractères!");
```

Détection des débordements (modes de compilation)

Voir aussi Stack canaries, ASLR et DEP/NX.

“Stack Buffer Overflow”

```
const std = @import("std");
```

```
fn foo(input: *const [8]u8) void {  
    var buffer: [8]u8 = undefined;  
    @memcpy(buffer[0..], input);  
}
```

```
pub fn main() void {  
    foo("Plus de 8 caractères!"); // 22 caractères  
}
```

expected type ‘const [8]u8’, found ‘const [22:0]u8’

“Double Free”

“Double Free”

```
var gpa = std.heap.DebugAllocator(.{}){};  
defer _ = gpa.deinit();  
const allocator = gpa.allocator();
```

“Double Free”

```
var gpa = std.heap.DebugAllocator(.{}){};  
defer _ = gpa.deinit();  
const allocator = gpa.allocator();
```

```
var number = try allocator.create(i32);  
allocator.destroy(number);
```

“Double Free”

```
var gpa = std.heap.DebugAllocator(.{}){};  
defer _ = gpa.deinit();  
const allocator = gpa.allocator();
```

```
var number = try allocator.create(i32);  
allocator.destroy(number);
```

```
allocator.destroy(number);
```


“Double Free”

```
var gpa = std.heap.DebugAllocator(.{}){};  
defer _ = gpa.deinit();  
const allocator = gpa.allocator();
```

```
var number = try allocator.create(i32);  
allocator.destroy(number);
```

```
allocator.destroy(number);
```

“Double Free”

```
var gpa = std.heap.DebugAllocator(.{}){};  
defer _ = gpa.deinit();  
const allocator = gpa.allocator();
```

```
var number = try allocator.create(i32);  
allocator.destroy(number);
```

```
allocator.destroy(number);
```

Solution idiomatique :

- typer avec un optional et mettre le pointeur à null.

 **ZIG et C**



Méta-programmation en ZIG



Comptime vs. Runtime

Code exécuté durant la compilation

Type universel

HKT et le Pure du Foncteur

```
fn Pure(M: fn (type) type, A: type) type {  
    return fn (A) M(A);  
}
```

```
fn Optional(T: type) type { return ?T; }
```

```
fn pure(A: type) Pure(Optional, A) {  
    return struct {  
        fn fun(v: A) Optional(A) { return v; }  
    }.fun;  
}
```

HKT et le Pure du Foncteur

```
fn Optional(T: type) type { return ?T; }
```

```
fn pure(A: type) Pure(Optional, A) {  
    return struct {  
        fn fun(v: A) Optional(A) { return v; }  
    }.fun;  
}
```


Conception logiciel et ZIG



Programmation Orientée Donnée

```
const Point = struct {  
    x: i32,  
    y: i32,  
  
    fn init(x: i32, y: i32) @This() {  
        return .{ .x = x, .y = y};  
    }  
  
    fn move(self: @This(), dx: i32, dy: i32) @This() {  
        return .{ .x = self.x + dx, .y = self.y + dy};  
    }  
};
```

Programmation Orientée Donnée

Function call or Static Dispatch?

```
fn main() void {  
    const zero = Point.init(0, 0);  
  
    // Classical functional call  
    const point1 = Point.move(zero, 42, 42);  
  
    // DOT syntax (syntactic sugar)  
    const point2 = zero.move(42, 42);  
}
```

Programmation Orientée Donnée

Types sommes et Filtrage de motif

```
const Number = union(enum) {  
    entier: i64, flottant: f64,  
  
    fn increment(self: @This()) @This() {  
        return switch (number) {  
            .entier => |n| .{ .entier = n + 1 },  
            .flottant => |f| .{ .flottant = f + 1 },  
        };  
    }  
};
```

Programmation Orientée Objet

Encodage à l'aide de table virtuelle

```
const Figure: type = struct {  
    v_impl: *anyopaque,  
    v_surface: *const fn (self: *anyopaque) f64,  
  
    pub fn surface(self: @This()) f64 {  
        return self.v_surface(self.v_impl);  
    }  
  
    pub inline fn from(impl: anytype) @This() { ... }  
};
```

Programmation Orientée Objet

Encodage à l'aide de table virtuelle

```
const Cercle = struct {  
    rayon: f64,  
  
    pub fn surface(self: @This()) f64 {  
        return std.math.pi * self.rayon * self.rayon;  
    }  
};
```

Instanciation : `Figure.from(&Cercle{ .rayon = 10 })`

Programmation par Modules

Definition de la signature de module `Stack`

```
fn Stack(comptime T:fn (type) type, comptime A:type) type {  
    return struct {  
        create: *const fn () T(A),  
        push: *const fn (Allocator, A, T(A)) anyerror!T(A),  
        peek: *const fn (T(A)) ?A,  
        pop: *const fn (Allocator, T(A)) Pair(?A, T(A)),  
    };  
}
```

Programmation par Modules

Mise en oeuvre du module `Stack` avec les `List`

```
fn StackList(comptime A: type) Stack(List, A) {  
    return .{  
        .create = struct {  
            fn fun() List(A) {  
                return .nil();  
            }  
        }.fun,  
        ...  
    };  
}
```


Programmation par Modules

Mise en pratique

Programmation par Modules

Mise en pratique

```
var heap = std.heap.DebugAllocator(.{}){};  
defer _ = heap.deinit();  
const allocator = heap.allocator();
```

Programmation par Modules

Mise en pratique

```
var heap = std.heap.DebugAllocator(.{}){};  
defer _ = heap.deinit();  
const allocator = heap.allocator();
```

```
const S = StackList(u32);  
const stack = try S.push(allocator, 1, S.create());  
defer S.deinit(allocator, stack);
```

Programmation par Modules

Mise en pratique

```
var heap = std.heap.DebugAllocator(.{}){};  
defer _ = heap.deinit();  
const allocator = heap.allocator();  
  
const S = StackList(u32);  
const stack = try S.push(allocator, 1, S.create());  
defer S.deinit(allocator, stack);  
  
try std.testing.expectEqual(1, S.peek(stack));
```

Programmation par Modules

Mise en pratique

```
var heap = std.heap.DebugAllocator(.{}){};  
defer _ = heap.deinit();  
const allocator = heap.allocator();  
  
const S = StackList(u32);  
const stack = try S.push(allocator, 1, S.create());  
defer S.deinit(allocator, stack);  
  
try std.testing.expectEqual(1, S.peek(stack));
```

Programmation Fonctionnelle

- Pas de fonction anonyme
- Pas de “closure”
- Données de type fonctionnel

```
const incr : *const fn(u32) u32 = struct {  
    fn fun(a:u32) u32 {  
        return a + 1;  
    }  
}.fun;
```

ZIG et la construction de binaires



Pourquoi choisir  ZIG ?



Ecosystème  **ZIG**



Conclusion et Perspectives

